

Plano e Matriz de Segurança da API – I-Stock Manager

Enrico F. Hoefel

1. Contexto e escopo

A I-Stock Manager API é uma API REST (Spring Boot, contrato OpenAPI 3.0 definido em API First) para o controle de estoque de pastilhas industriais da DDA Usinagem Industrial. Ela expõe os recursos Autenticação, Pastilhas, Fabricantes, Fornecedores, Motivos de Movimentação, Movimentações, Usuários, Alertas, Dashboard e Relatórios, com autenticação por token JWT (Bearer) e três perfis de acesso: ADMINISTRADOR, GESTOR e OPERADOR.

Este documento registra as decisões de segurança que serão implementadas com Spring Security. Ele contém a Matriz de Segurança (seção 2), o Plano de Segurança (seção 3) e a definição de perfis e permissões (seção 4).

2. Matriz de Segurança

Recurso/Operação	Ameaça/Vulnerabilidade	Possível impacto	Controle preventivo
POST /auth/login	Força bruta e credential stuffing	Acesso não autorizado ao sistema	Limite de tentativas com bloqueio temporário, mensagem de erro genérica (401) sem indicar se o e-mail existe, senhas fortes (mínimo de 8 caracteres)
Armazenamento de credenciais	Vazamento do banco de dados	Exposição das senhas dos usuários	Hash com BCrypt (salt automático); a senha nunca é retornada pela API nem registrada em logs
Token JWT	Roubo, forjamento ou reutilização do token	Sequestro de sessão e acesso indevido	Assinatura validada em toda requisição, segredo forte fora do código (variável de ambiente), expiração curta (campo expiraEm), tráfego somente via HTTPS
Todos os endpoints, exceto login	Acesso sem autenticação	Exposição ou alteração indevida de dados de estoque	Regra anyRequest().authenticated(); resposta 401 para token ausente ou inválido

Recurso/Operação	Ameaça/Vulnerabilidade	Possível impacto	Controle preventivo
Endpoints por perfil (ex.: POST /usuarios, DELETE /pastilhas/{id})	Escalonamento de privilégio (perfil sem permissão chamando operação restrita)	Criação de usuários indevidos, exclusão ou alteração de dados críticos	Autorização por perfil (RBAC) com hasRole e @PreAuthorize; resposta 403; princípio do menor privilégio
/usuarios e /usuarios/{id}	Exposição de dados pessoais e manipulação de contas	Vazamento de e-mails, alteração de perfis	Acesso restrito ao ADMINISTRADOR; DTOs de resposta sem senha ou hash; exclusão lógica
Cadastro e atualização (POST e PUT de pastilhas, fabricantes, fornecedores, motivos, usuários)	SQL Injection	Manipulação, vazamento ou perda de dados	Spring Data JPA com consultas parametrizadas, sem concatenação de SQL; validação com Bean Validation (@Valid) conforme os limites do contrato
Filtros de busca (GET /pastilhas?codigo=&descricao=/movimentacoes, /fabricantes?nome=)	SQL Injection via parâmetros de busca parcial	Acesso indevido a dados do banco	Parâmetros nomeados (JPQL ou Specifications), tipagem dos parâmetros (datas, inteiros, enums), lista fixa de campos de ordenação
Campos de texto (descrição, nome, observação)	XSS armazenado	Execução de script no navegador de quem visualiza o dado	Validação de tamanho e formato, rejeição de HTML nos campos de texto, escape na saída pelo front-end, cabeçalhos X-Content-Type-Options e Content-Security-Policy
Requisições alteradoras (POST, PUT, DELETE)	CSRF	Operações executadas em nome do usuário autenticado	API stateless com token no cabeçalho Authorization (sem cookie de sessão), SessionCreationPolicy.STATELESS; CSRF desabilitado com essa justificativa (se o token passar a ser armazenado em cookie, o CSRF deve ser reativado)
Chamadas do front-end ao navegador	CORS permissivo	Sites de terceiros consumindo a API com o token do usuário	CORS com lista explícita de origens, métodos e cabeçalhos permitidos; sem uso de "*"
PUT /pastilhas/{id}	Mass assignment (alterar saldoAtual pelo corpo da requisição)	Saldo de estoque adulterado	DTOs de entrada separados da entidade (PastilhaInput não possui saldoAtual); saldo alterado somente por movimentações

Recurso/Operação	Ameaça/Vulnerabilidade	Possível impacto	Controle preventivo
POST /movimentacoes	Adulteração de saldo, saída acima do saldo, concorrência entre requisições	Estoque inconsistente e decisões de compra erradas	Quantidade mínima 1; bloqueio de saldo negativo (409); transação (@Transactional) com controle de concorrência; idUsuario e dataHora definidos pelo servidor, nunca pelo cliente
Movimentações (histórico)	Repúdio e alteração do histórico	Perda de rastreabilidade de quem movimentou o estoque	Movimentações imutáveis (sem PUT/DELETE no contrato); registro do usuário autenticado em cada movimentação
DELETE de pastilhas, fabricantes, fornecedores e motivos	Exclusão indevida de dados	Perda de histórico e quebra de vínculos	Exclusão lógica (campo ativo), restrita ao ADMINISTRADOR; retorno 409 quando há vínculos
Relatórios e Dashboard	Acesso indevido a informações estratégicas de consumo e reposição	Exposição de dados sensíveis do negócio	Relatórios restritos a GESTOR e ADMINISTRADOR; período obrigatório e validado
Listagens paginadas	Abuso de volume (negação de serviço)	Lentidão ou indisponibilidade da API	Paginação obrigatória com tamanho máximo de 100 itens; limite de requisições por cliente quando aplicável
Tratamento de erros	Vazamento de informações internas (stack trace, estrutura do banco)	Facilita ataques posteriores	Tratador global de exceções (@RestControllerAdvice) com resposta padronizada (schema Erro), sem detalhes internos
Transporte e configuração	Interceptação de tráfego, segredos expostos, componentes vulneráveis	Vazamento de tokens e credenciais	HTTPS/TLS e HSTS em produção; segredos em variáveis de ambiente; dependências atualizadas; documentação Swagger desabilitada em produção

3. Plano de Segurança da API

3.1 Autenticação

A autenticação será feita por e-mail e senha em POST /auth/login. Em caso de sucesso, a API emite um token JWT assinado, com expiração curta (por exemplo, 1 hora), contendo o identificador e o perfil do usuário. O cliente envia esse token em todas as requisições

seguintes no cabeçalho Authorization: Bearer. Um filtro (OncePerRequestFilter) valida assinatura e validade do token antes de cada requisição e carrega o usuário no contexto do Spring Security. A API não mantém sessão no servidor (stateless).

3.2 Recursos públicos e protegidos

Somente POST /auth/login é público. Todos os demais endpoints exigem autenticação e são protegidos conforme o perfil. A documentação interativa (Swagger UI) fica disponível apenas no ambiente de desenvolvimento.

3.3 Perfis de acesso

- **ADMINISTRADOR:** acesso total, incluindo gestão de usuários e inativação de registros.
- **GESTOR:** gerencia o cadastro de pastilhas, fabricantes, fornecedores e motivos, registra movimentações e consulta relatórios.
- **OPERADOR:** consulta o catálogo, os alertas e o dashboard e registra movimentações de entrada e saída.

3.4 Controle de autorização

A autorização será feita por perfil (RBAC), em duas camadas: regras por rota e método no SecurityFilterChain (requestMatchers com hasRole) e anotações @PreAuthorize nos métodos, com @EnableMethodSecurity. Qualquer requisição sem permissão recebe 403; sem autenticação, 401. A regra padrão é negar o que não foi explicitamente liberado.

3.5 Armazenamento e proteção das credenciais

As senhas são armazenadas apenas como hash BCrypt (BCryptPasswordEncoder). A senha é recebida somente na criação do usuário e no login, nunca é retornada pela API e nunca aparece em logs. O segredo de assinatura do JWT e as credenciais do banco ficam em variáveis de ambiente, fora do repositório.

3.6 Configuração do CORS

O CORS será configurado com CorsConfigurationSource no Spring Security, com as seguintes regras:

- origens permitidas: apenas a origem do front-end (uma por ambiente, por exemplo http://localhost:5173 em desenvolvimento), sem uso de "*";
- métodos: GET, POST, PUT, DELETE e OPTIONS;
- cabeçalhos: Authorization e Content-Type;
- credenciais (cookies) desabilitadas, pois a autenticação usa o cabeçalho Authorization;

- tempo de cache do preflight definido (maxAge).

3.7 Outros mecanismos de mitigação

- **SQL Injection:** consultas parametrizadas (JPA), sem concatenar entrada do usuário em SQL.
- **XSS:** validação das entradas de texto, escape na saída pelo front-end e cabeçalhos de segurança (Content-Security-Policy, X-Content-Type-Options).
- **CSRF:** desabilitado de forma justificada, por a API ser stateless e usar token no cabeçalho.
- **Validação:** Bean Validation em todos os DTOs de entrada, respeitando os limites já definidos no contrato (tamanhos, mínimos, CNPJ com 14 dígitos).
- **Integridade do estoque:** saldo alterado apenas por movimentações, em transação, sem permitir saldo negativo.
- **Rastreabilidade:** movimentações imutáveis, com o usuário autenticado registrado.
- **Erros e logs:** respostas de erro padronizadas, sem detalhes internos, e logs sem dados sensíveis.
- **Transporte:** HTTPS em produção, com HSTS.

4. Perfis e permissões

Recurso/Operação	Público	Operador	Gestor	Administrador
Realizar autenticação (POST /auth/login)	Sim	Sim	Sim	Sim
Consultar pastilhas, fabricantes, fornecedores e motivos (GET)	Não	Sim	Sim	Sim
Consultar alertas e dashboard (GET)	Não	Sim	Sim	Sim
Consultar histórico de movimentações (GET)	Não	Sim	Sim	Sim
Registrar movimentação de entrada ou saída (POST /movimentacoes)	Não	Sim	Sim	Sim
Cadastrar e alterar pastilhas, fabricantes, fornecedores e motivos (POST e PUT)	Não	Não	Sim	Sim
Associar ou remover fornecedor de uma pastilha	Não	Não	Sim	Sim

Recurso/Operação	Público	Operador	Gestor	Administrador
Inativar pastilhas, fabricantes, fornecedores e motivos (DELETE)	Não	Não	Não	Sim
Consultar relatórios de consumo e reposição	Não	Não	Sim	Sim
Gerenciar usuários (listar, cadastrar, alterar e inativar)	Não	Não	Não	Sim

5. Ajustes sugeridos ao contrato OpenAPI

Para manter o contrato coerente com este plano, na implementação serão acrescentadas as respostas 403 (perfil sem permissão) aos endpoints restritos e 429 (excesso de tentativas) ao login.